CS1200: Intro. to Algorithms and their Limitations Profs. Anshu and Nesson

Problem Set 3

Harvard SEAS - Fall 2026 *Due: Thu Oct. 6, 2026 (11:59pm)*

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.' Please remember to select pages when you submit on Gradescope. A problem set on the border between two letter grades cannot be rounded up if pages are not selected.

Your name: Andrew Jing

Collaborators and External Resources: Claude Code helped with LaTeX formatting, and also helped guide me through some big-O simplifications

No. of late days used on previous psets: 1

No. of late days used after including this pset: 1

1. (Choosing Algorithms and Data Structures)

   Suppose the US Census Bureau was going to develop a new database to keep track of the exact ages of the entire US population, and publish statistics on it. The data it has on each person is an exact birthdate `bday` (year, month, and date) and a unique identifier `id` (e.g. social security number — pretend that these are assigned at birth). Make sure to read the question clearly as each subpart is a different situation.

   For each of the three (optionally, four) scenarios below,

   (i) Select the best algorithm or data structure for the Census Bureau to use from among the following:

   - sorting and storing the sorted dataset
   - storing in a binary search tree (balanced and possibly augmented)
   - storing in a hash table
   - running randomized QuickSelect.

   (ii) Explain how you would use the algorithm or data structure (including any necessary augmentations) to solve the stated problem, For instance, what would you take as keys and values, what updates and queries (in case you use a dynamic data structure) would you issue, and how you would read off the results to obtain the desired statistics. If you

are using an augmented data structure, specify the augmentation you are making and if you are using a hash table, specify its size.

(iii) State what the runtime would be as a function of all of the relevant parameters: the size n of the US population being surveyed, the number u of updates issued at the specified time intervals, and/or the number s of statistics released at the specified time intervals. (These parameters are not all freely varying in the parts below, s may be a fixed constant or a function of n; state any such constraints in your answer)

In each scenario, you should assume (unrealistically) that the described queries or statistics are the *only* way in which the data is going to be used, so there is no need to support anything else. If you find that only `id` or `bday` is relevant or necessary for some part, you can feel free to ignore the other component in your data structure.

Example (Decennial Quartiles): Every ten years as part of the Decennial Census, the Bureau obtains a fresh list of (`id`, `bday`) pairs for the entire US population and wishes to release the 25th, 50th, and 75th percentiles of the population ages (to the day).

Solution:

(i) **Recommendation:** I would recommend that the Census Bureau use `QuickSelect`. I guess that a large enough of a fraction of the US population changes every 10 years that processing the updates individually would be more expensive than doing batch computations.

(ii) **How I would use my data structure/ algorithm:** They should use the `bday`'s as keys; the `id`'s are not needed because we only want to release age percentiles. Each time they should run `QuickSelect` with ranks $i = n/4, n/2, 3n/4$, where n is the size of the population at the time.

(iii) **Runtime:** We are calculating $s = 3$ statistics every 10 years, and each execution of `QuickSelect` takes expected time $O(n)$, for a total runtime of $s \cdot O(n) = O(n)$.

(a) (Reporting Age Rankings)

Every ten years as part of the Decennial Census, the Bureau collects a fresh list of (`id`, `bday`) pairs from the entire US population. (It does not reuse data from the previous Decennial Census, so everyone is re-surveyed.) In order to incentivize participation, the Bureau promises to tell every respondent their age-ranking in the population after the survey is done (e.g. "you are the 796,421'th oldest person among those who responded to the Census"). If two people have the same birthday, its ok if their age-ranking is arbitrarily ranked with respect to each other.

Solution.

(i) **Recommendation:** I'd recommend sorting+storing. Because the dataset is inputted up front, rather than updating some previous dataset, we can just sort the

whole thing. Also, because everyone gets a ranking, we need a full sorting. I'm assuming that the Bureau sends everyone their ranking once, rather than allowing people to query their ranking on demand.

(ii) **How I would use my data structure/algorithm:** We're ranking by age, so `bday` are the keys and `id` are the values. After sorting, the oldest person is at index 0 and the youngest is at $n-1$, so for each sorted entry at index i, we tell the person with $\texttt{id}_i$ they are the $i+1$th oldest in the survey. The sorting algorithm can be any $O(n \log n)$ algorithm, since we can rank duplicate ages arbitrarily relative to each other.

(iii) **Runtime:** Since we don't know what the tuples will be used for and only know the incentives, we sort n tuples and report $s = 1$ statistics to n people, with each report taking constant time. That makes $O(n \log n) + s \cdot n \cdot O(1) = O(n \log n) + O(n) = O(n \log n)$ total runtime.

(b) (Daily Quartiles)

After each day, the Bureau obtains a list of data (given as a list of (`id`, `bday`) pairs) to add to or remove from its database due to births, deaths, and immigration, and publishes an updated 25th, 50th, and 75th percentile of the population ages. *HINT:* If we are making changes to the data, would a static or dynamic data structure be best?

Solution.

(i) **Recommendation:** Since we're not told how to handle invalid inputs, let's assume that every to-be-deleted tuple actually has a matching entry already in the database, and that every `id` has at most one entry. Because of that data correctness assumption, we can skip data validation steps that involve `id`, meaning that we don't need `id`. I suggest just using a size-augmented balanced BST. This is because we need full sorting and dynamic support to efficiently recalculate quartiles every day. This approach works because our BST allows duplicate keys.

(ii) **How I would use my data structure/algorithm:** The size-augmented balanced BST should have `bday` as keys and have no values. We preprocess the existing database's n entries using their birthdays. Every day, we insert or delete u birthdays in the update list. Afterwards, we walk the BST 3 times in search of the $k = \lfloor n'/4 \rfloor$th, $\lfloor n'/2 \rfloor$th, and $\lfloor 3n'/4 \rfloor$th smallest keys (0-indexed), where n' is the tree size after all of the day's updates. We achieve this by initializing variable i to the size of the root's left child (0 is null) and returning `bday` if $i = k$. If $i > k$, recurse on the left subtree, and if $i < k$ recurse on the right subtree but subtract $i+1$ from k.

(iii) **Runtime:** A size-augmented balanced BST's preprocessing is $O(n \log n)$, and each insert or delete runs in $O(\log(n+u-1))$, since each insert/delete happens on a tree with size at most $n+u-1$ (when all previous updates were inserts). Finally, we have $s = 3$ tree walks with $O(\log(n+u))$ runtime. So in total we have an $O(n \log n)$ preprocess cost and a runtime of $u \cdot O(\log(n+u-1)) + s \cdot O(\log(n+u)) = O(u \log(n+u) + 3 \log(n+u)) = O((u+3) \log(n+u))$ per day. Assuming $0 < u \leq n$, the runtime is now $O(u \log n)$ every day. For daily runtime, n refers to the size of the dataset just before the day's updates, not just the number of entries at day 0, and same for the next problems.

(c) (Age Lookups)

For privacy reasons, the Bureau decides to not publish any statistics on the population ages, but just wants to maintain a database where the age of any member of the population can be looked up quickly, and the database can be quickly updated daily according to births, deaths, and immigration.

Solution.

(i) **Recommendation:** This should be a hash map because we don't have to sort anything and want quick (expected) lookups and updates.

(ii) **How I would use my data structure/algorithm:** The hash map should have keys `id` and values `bday`. Its number of hash buckets m depends on the Bureau's desired storage-to-speed tradeoff, but m should be linearly proportional to the current total size of the hash map so let's say max load factor is 1 and we start with $m = 2n$. Updating means inserting $(\texttt{id}, \texttt{bday})$ or deleting `id`. Again, without a stated policy we must assume that each to-be-deleted tuple has a matching one in the database, and that each `id` has at most one entry.

(iii) **Runtime:** That means a preprocessing costs $O(m) + n \cdot O(1) = O(n)$ for storage reservation and n hashes+stores, and we bound daily updates to $u \cdot O(1) = O(u)$ expected runtime. This retains lookup support with expected runtime of $O(1+1) = O(1)$ since our max load factor is 1.

(d) (Daily Quartiles, optional[1]) The Bureau receives a daily list of updates and publish the 25th, 50th, and 75th percentile of population ages, similar to (b). This time, however, when an update deletes someone's data (i.e. death, emigration), the Bureau only receives their `id`. Incoming data (i.e. birth, immigration) is still in the format $(\texttt{id}, \texttt{bday})$.

Solution.

(i) **Recommendation:** I'm just gonna let myself use multiple data structures here, since otherwise runtime would suffer. We use one hash map and one balanced BST. This trades $O(n)$ more storage for asymptotically faster queries.

(ii) **How I would use my data structure/algorithm:** The balanced BST uses `bday` as keys and has no values, and the hash map uses `id` for keys and `bday` for values with a max load factor of 1 and a starting $m = 2n$. After preprocessing everyone, each update goes like this: (1) inserting $(\texttt{id}, \texttt{bday})$ means checking that `id` doesn't exist in the hash map yet, inserting `bday` into the BST, then inserting $(\texttt{id}, \texttt{bday})$ into the hash map, and (2) deleting `id` means searching the hash map for `id` and making sure it exists, then deleting the corresponding `bday` from the BST, and then deleting `id` from the hash map. At the end of each day, we use the same strategy as 1b to report quantiles.

(iii) **Runtime:** Preprocessing takes $O(n)+O(n \log n) = O(n \log n)$ for the hash map and BST. Each update takes an expected $O(1)+O(\log(n+u)+O(1)) = O(\log(n+u))$ to validate `id`, insert or delete from the BST, and then insert or delete from the hash map. We have $s = 3$ stats to report, with each taking $O(\log(n+u))$. This means that daily expected runtime is $u \cdot O(\log(n+u)) + 3 \cdot O(\log(n+u)) = O((u+3) \cdot \log(n+u))$. With $0 < u \leq n$, this simplifies to $O(u \cdot \log n)$.

[1] This problem won't make a difference between N, L, R-, and R grades.

2. (Reflection) In what ways do LLMs impact your learning outcomes for this course? If you use LLMs in learning course material, give some example(s) of how they helped you learn something that lectures/sections etc did not. If you do not use LLMs, discuss reasons for why you do not use them. All opinions are valid.

   Solution. LLMs allow me to ask short and quick questions without the cost of peppering a CA with questions that I also have to drive to campus for. For example, "why is my LaTeX not compiling?" is something I'd never want to debug with a person, and asking LLMs just saves me and everyone else a bunch of time without taking away from core learning. Also, they're a great sanity check to chat with and see if I made any impactful typos. Basically, using LLMs allows me to learn more by outsourcing the annoying-but-necessary stuff, almost like not having to remember all your friends' phone numbers and outsourcing that to an app.

3. Once you're done with this problem set, please fill out this survey so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses!